

Car Rental Management System

Business Requirements & Technical Design Document

A hands-on Salesforce learning project built entirely inside a single Developer Org

Platform: Salesforce (Lightning Platform)

Prepared for: Shivam | Final-year IT student, KIIT

Document type: Business Requirements Document (BRD)

Scope note: No external APIs, payment gateways, Named Credentials, or third-party integrations. Everything runs inside one Salesforce Developer Org.

1. Executive Summary

The **Car Rental Management System** is a self-contained Salesforce application that simulates how a small car-rental company would manage its fleet, customers, and bookings inside the Salesforce platform. The project is designed as a structured, end-to-end learning exercise: it takes concepts already learned at a basic level (objects, fields, security, automation, SOQL, Apex) and applies them together in one realistic build, adding Lightning Web Components, Apex triggers, Flow automation, and an Approval Process on top.

The application lets a Rental Agent search available cars through a custom LWC interface, estimate rental pricing, apply coupons, and create bookings. High-value bookings route through a Manager approval step. Car availability, customer statistics, and coupon usage update automatically through Flow and Apex, and the system enforces business rules such as license validity and booking-date overlap prevention.

The build is intentionally scoped to **intermediate-beginner level** and excludes all external integrations (payment gateways, maps, SMS/email APIs, Named Credentials, OAuth) so it can be completed entirely within one Developer Org, with every design decision explained in terms of Salesforce architectural trade-offs.

2. Project Objectives

- Reinforce Salesforce fundamentals (objects, fields, relationships, page layouts, record types) through a single realistic build.
- Gain hands-on experience with the full security model: Profiles, Permission Sets, OWD, Roles, Sharing Rules.
- Apply declarative automation correctly: Record-Triggered Flow, Scheduled Path, Approval Process, Validation Rules, Formula Fields.
- Build proficiency in SOQL: filtering, ordering, relationship queries, aggregate/GROUP BY queries.
- Develop core Apex skills: Trigger + Trigger Handler pattern, bulkification, governor-limit-safe code, Apex test classes.
- Build a multi-component Lightning Web Component application with parent-child communication and Apex integration.
- Understand *when* to use Flow vs. Apex vs. Formula vs. Validation Rule, not just how to implement each.
- Produce reports and a dashboard that give the business (a fictional rental company) usable operational insight.

3. Scope

3.1 In Scope

- Custom objects: Car, Customer, Booking, Coupon and their relationships.
- Declarative configuration: page layouts, record types, validation rules, formula fields, list views.
- Security model with 4 distinct users (Admin, Rental Agent, Rental Manager, Rental Viewer).
- Automation: Record-Triggered Flows, a Scheduled Path, and a Booking Approval Process.
- Apex: BookingTrigger + BookingTriggerHandler (overlap prevention), CarRentalController for LWC, test classes.
- A multi-component LWC car-search and booking experience (carRentalSearch and children).
- Reports, a dashboard, and curated list views for operational visibility.
- Sample data: 10+ cars, 5+ customers, 3 coupons, 10+ bookings across all statuses.

3.2 Out of Scope

- External APIs, payment gateways, Google Maps, Twilio, or email-sending services.
- Named Credentials, OAuth integrations, MuleSoft, Heroku, or any external database.
- Third-party managed packages.
- A public-facing customer website or portal (Experience Cloud) — the app is internal-only; agents book on the customer's behalf.
- Real payment processing — Payment_Status__c is tracked as a status field only.

4. User Personas & Access Model

User / Profile	Purpose	Can Do	Cannot Do
Car Rental Admin System Administrator profile	Full application & org control	Create/edit/delete Cars, Customers, Bookings; manage Coupons, users, configuration; view all data/reports	—
Rental Agent Custom profile + Permission Sets	Front-desk booking creation	View/search cars; create/view customers; create & edit own bookings; cancel bookings; apply coupons	Delete cars/customers/bookings; modify coupons or configuration
Rental Manager Custom profile	Fleet oversight & approvals	View/edit cars; view all customers & bookings; approve/reject bookings; view reports & dashboards	Modify org-wide configuration
Rental Viewer Custom profile, read-only	Management visibility	View Cars, Customers, Bookings, Reports, Dashboards	Create, edit, or delete any business record

5. Data Model Overview

Four custom objects form the core data model. **Booking__c** is the central transactional object, linked to Car__c, Customer__c, and Coupon__c via Lookup relationships (chosen over Master-Detail so bookings can be deleted/archived independently and can carry their own sharing rules — critical since Booking__c OWD is Private).

5.1 Car__c — Fleet Inventory

Field API Name	Type	Purpose
Car_Model__c	Text	Model name, e.g. Tata Nexon
Brand__c	Picklist	Tata / Hyundai / Maruti / Mahindra / Toyota / Honda / Kia / Other
Car_Type__c	Picklist	Hatchback / Sedan / SUV / MUV
Fuel_Type__c	Picklist	Petrol / Diesel / CNG / Electric / Hybrid
Transmission__c	Picklist	Manual / Automatic
Seats__c	Number	Passenger capacity
Daily_Rental_Rate__c	Currency	Current daily rate
Rating__c	Number (0-5)	Customer rating
Location__c	Picklist	City where the car is based
Status__c	Picklist	Available / Booked / Maintenance / Inactive
Registration_Number__c	Text (Unique)	Legal registration ID
Car_Image_URL__c	URL	Image reference for LWC UI only
Total_Bookings__c	Number	Maintained via automation
Active__c	Checkbox	Fleet-active flag

5.2 Customer__c

Field API Name	Type	Purpose
Email__c / Phone__c	Email / Phone	Contact details
Driving_License_Number__c	Text (Unique)	License identifier
Driving_License_Expiry__c	Date	Validated against booking Start_Date__c
Customer_Status__c	Picklist	Active / Suspended / Inactive
Total_Bookings__c / Total_Spent__c	Number / Currency	Maintained via Flow on booking completion

5.3 Booking__c — Central Transactional Object

Field API Name	Type	Purpose
Customer__c / Car__c / Coupon__c	Lookup	Relates the booking to its participants
Start_Date__c / End_Date__c	Date	Rental period
Number_of_Days__c	Formula	End Date - Start Date + 1
Daily_Rate__c	Currency	Snapshot of the car's rate at booking time (not linked live to Car)
Base_Rental_Cost__c	Currency/Formula	Daily_Rate__c x Number_of_Days__c
Discount_Percentage__c / Discount_Amount__c	Percent / Currency	Coupon-driven discount
Tax__c / Total_Amount__c	Currency	Fixed-rate tax; final payable amount
Status__c	Picklist	Draft / Pending Approval / Confirmed / Active / Completed / Cancelled / Rejected
Payment_Status__c	Picklist	Not Paid / Paid / Refunded (tracked only, no real processing)
Approval_Required__c / Approved_By__c / Approval_Date__c	Checkbox / Lookup(User) / DateTime	Approval trail

5.4 Coupon__c

Field API Name	Type	Purpose
Discount_Percentage__c	Percent	Discount applied
Start_Date__c / End_Date__c	Date	Validity window
Maximum_Uses__c / Used_Count__c	Number	Usage cap, incremented via Flow on confirmation
Minimum_Booking_Amount__c / Maximum_Discount__c	Currency	Eligibility & discount cap
Active__c	Checkbox	Master on/off switch

6. Security Model

Organization-Wide Defaults start restrictive so the project can meaningfully demonstrate Roles, Permission Sets, and Sharing Rules rather than relying on implicit full access.

Object	OWD	Rationale
Car__c	Public Read Only	Every internal user should see the fleet; only Admin/Manager should edit
Customer__c	Private	Customer data is sensitive; visibility granted via Role Hierarchy / Sharing Rules
Booking__c	Private	Enables demonstrating record ownership, manual sharing, and Approval-driven access
Coupon__c	Public Read Only	All agents must be able to validate a coupon; only Admin manages them

6.1 Layered Access Strategy

- **Profiles** set the baseline object/field-level permissions per role (Agent, Manager, Viewer).
- **Permission Sets** grant narrow, additive capabilities (e.g. "Apply Coupon") without duplicating profiles.
- **Role Hierarchy** lets Managers see records owned by Agents beneath them, without loosening OWD globally.
- **Sharing Rules** extend access declaratively (e.g. all Agents can view all Active bookings) where hierarchy alone is insufficient.
- Approval Process reassigns record ownership/visibility to the Manager during the approval step.

7. Automation Layer & Decision Rationale

A core learning goal of this project is choosing the *right* tool for each piece of logic. The table below is the running design decision log referenced throughout the build.

Requirement	Tool Used	Why
Booking Duration	Formula Field	Pure declarative calculation, no DML needed, always in sync
Base Cost / Discount / Tax roll-up	Record-Triggered Flow	Multi-field sequential calculation with lookups to Coupon — cleaner in Flow than a formula chain
End Date >= Start Date; Cancellation reason required	Validation Rule	Simple field-level checks with no cross-object queries
Prevent overlapping bookings	Apex Trigger	Requires querying other Booking records for the same Car — Flow cannot safely bulk-query and compare date ranges at this complexity without risking limits
Booking > █20,000 needs approval	Approval Process	Native human-approval workflow with audit trail, reassignment, and email/task notification

Requirement	Tool Used	Why
Car status / Customer stats on completion	Scheduled Path (Flow)	Needs to react to elapsed time (End Date passing), not just a save event
LWC search & booking creation	Apex Class (@AuraEnabled)	LWC cannot query records directly — needs a controlled, cacheable server-side API

8. Lightning Web Component Architecture

The user-facing layer is a component tree rooted at **carRentalSearch**, split into small, single-purpose children rather than one large component — mirroring good separation of concerns and making event communication (parent to child via `@api`, child to parent via `CustomEvent`) an explicit teaching point.

Component	Role
carRentalSearch	Parent container; owns search state and orchestrates Apex calls
carFilterPanel	Location, dates, seats, rate, rating, transmission, fuel-type filters
carList	Renders result set; handles loading/empty/error states
carCard	Single car summary card with Estimate/Book Now actions
carDetail	Expanded car view on selection
bookingForm	Customer, coupon, and date capture; triggers server-side calculation
bookingSummary	Read-only computed pricing breakdown before submission

8.1 Server-Side Filtering

Filtering happens through Apex/SOQL rather than client-side JavaScript over a full car list — this keeps the component scalable and demonstrates proper LWC-to-Apex data flow (LWC → Apex Controller → SOQL → Salesforce).

9. End-to-End Booking Process

- Rental Agent opens the Car Rental Management app and the car-search LWC.
- Agent enters location, dates, and filters; matching available cars are returned via SOQL.
- Agent selects a car and clicks Estimate Booking to preview pricing.
- Agent clicks Book Now, selects/creates a Customer, and optionally enters a coupon code.
- System validates dates, license validity, car availability/overlap, and coupon eligibility.
- System calculates rental days, base cost, discount, tax, and final total.
- If `Total_Amount__c ≤ $20,000` → booking is Confirmed directly and the car is marked Booked.
- If `Total_Amount__c > $20,000` → booking enters Pending Approval and routes to the Rental Manager.
- Manager approves (→ Confirmed, car → Booked) or rejects (→ Rejected).
- After the rental period ends, a Scheduled Path marks the booking Completed, frees the car, and updates customer statistics.

10. Non-Functional Requirements & Governance

- **Bulkification** — BookingTrigger must handle bulk insert/update without SOQL or DML inside loops.
- **Governor limits** — implementation must respect SOQL query limits, DML row limits, CPU time, and heap size.
- **Test coverage** — BookingTriggerHandlerTest and CarRentalControllerTest cover positive, negative, and bulk scenarios with meaningful assertions (not coverage-only tests).
- **Error handling** — user-facing messages are friendly (e.g. "This car is already booked for the selected dates."); raw Apex exceptions are never exposed.
- **No external dependencies** — the entire system must run inside a single Salesforce Developer Org.

11. Reports & Dashboard

Report	Purpose
All Cars (by Car Type)	Fleet composition overview
Available Fleet	Cars currently bookable
Bookings by Status	Pipeline visibility across the booking lifecycle
Revenue Report	Total and monthly revenue
Top Customers	Ranked by Total_Spent__c
Most Rented Cars	Ranked by booking count
Cancelled Bookings	Cancellation reasons for operational review

These feed a single **Car Rental Management Dashboard** summarizing fleet status, booking volume, revenue trends, and top performers — built for the Rental Manager persona.

12. Development Roadmap (Phased Delivery)

Phase	Deliverables
1. Project Setup	Lightning App, custom objects, fields, relationships, sample data
2. Configuration	Page layouts, record types, validation rules, formula fields, list views
3. Security	Profiles, Permission Sets, OWD, Roles, Sharing Rules, 4 test users
4. Basic Automation	Record-triggered Flows, Scheduled Path, customer stats, car status, coupon usage
5. Approval Process	Booking approval, Manager routing, approve/reject automation
6. SOQL	Guided query-writing exercises before advanced Apex
7. Apex	Apex classes, BookingTrigger + Handler, bulkification, error handling
8. LWC	Search, filters, car cards/detail, booking form, event-driven communication
9. Reports & Dashboard	All 7 reports plus the management dashboard

Phase	Deliverables
10. Testing	Functional, security, Flow, Approval, Trigger, Apex, LWC, and edge-case testing

Each phase follows a mentor-style loop: objective → concept explanation → exact build instructions → a small task for hands-on implementation → review → advance to the next phase only once complete.

13. Final Acceptance Criteria

- Rental Agent can search, filter, and view cars through the LWC with server-side filtering.
- Bookings auto-calculate price, apply valid coupons, and reject invalid coupons/dates/expired licenses.
- Apex trigger prevents overlapping bookings for the same car and is fully bulkified.
- High-value bookings correctly route through the Approval Process; Manager can approve or reject.
- Car status and customer statistics update automatically on booking confirmation/completion.
- All four personas (Admin, Agent, Manager, Viewer) demonstrate correctly differentiated access.
- Reports and dashboard render accurate, live operational data.
- Apex test classes pass with meaningful positive, negative, and bulk assertions.
- No external integrations of any kind are present in the final build.

14. Key Architectural Principles Applied Throughout

- **Lookup vs. Master-Detail** — Booking uses Lookups to Car/Customer/Coupon to preserve independent record ownership and Private sharing; Master-Detail is reserved for a case where cascade-delete and roll-up summaries genuinely add value.
- **Formula vs. Flow vs. Apex** — formulas for stateless per-record math, Flow for multi-step record automation and cross-object updates, Apex reserved for logic needing bulk querying/comparison (booking overlap) or LWC data services.
- **Validation Rule vs. Apex** — simple field-level checks stay declarative; only checks requiring queries against other records move to Apex.
- **Profile vs. Permission Set** — Profiles define the baseline per persona; Permission Sets layer on narrow, optional capabilities so profiles stay simple and reusable.
- **OWD + Role Hierarchy + Sharing Rules** — combined deliberately so each security layer's individual effect can be observed and tested.